NetBird 連線架構

peer 盡量直接用 WireGuard 連，server 幫 peer 找到彼此，打不通才轉送

各個 server 在做什麼

- management：記錄 peer、使用者、存取規則，把網路地圖推給 peer（gRPC）
- 登入（Dex，`/oauth2`）：發 token 給 dashboard 和 `netbird up`
- dashboard：靜態網頁，資料打 management 的 `/api`
- signal：peer 連上之前交換彼此的 IP 和 port，內容加密（gRPC）
- STUN：告訴 peer 它在 NAT 外面的位址（UDP 3478）
- relay：打不通時轉送 WireGuard 封包，唯一會傳真正資料的（WebSocket 或 QUIC，不是 gRPC）

API 和登入是 management 的一部分，跟 gRPC 共用同一個 port。合併版的 netbird-server 裡 management、signal、relay、STUN 都在同一個 process

接法

兩種接法的 dashboard 和 `netbird` 兩條一樣，差在 relay 走哪裡。origin 是 CloudFront 背後實際去要資料的位址

A：relay 走 CloudFront

```
dashboard
  browser ─HTTPS─▶ dashboard cf ─HTTP :80, no cert─▶ EIP ─▶ dashboard :80

management, signal (gRPC)
  peer ─gRPC─▶ mgmt cf ─HTTPS :443, LE─▶ EIP ─▶ netbird-server :443

API, login (HTTPS)
  browser ─HTTPS─▶ mgmt cf ─HTTPS :443, LE─▶ EIP ─▶ netbird-server :443

relay (WebSocket, not gRPC)
  peer ─WebSocket─▶ mgmt cf ─HTTPS :443, LE─▶ EIP ─▶ netbird-server :443

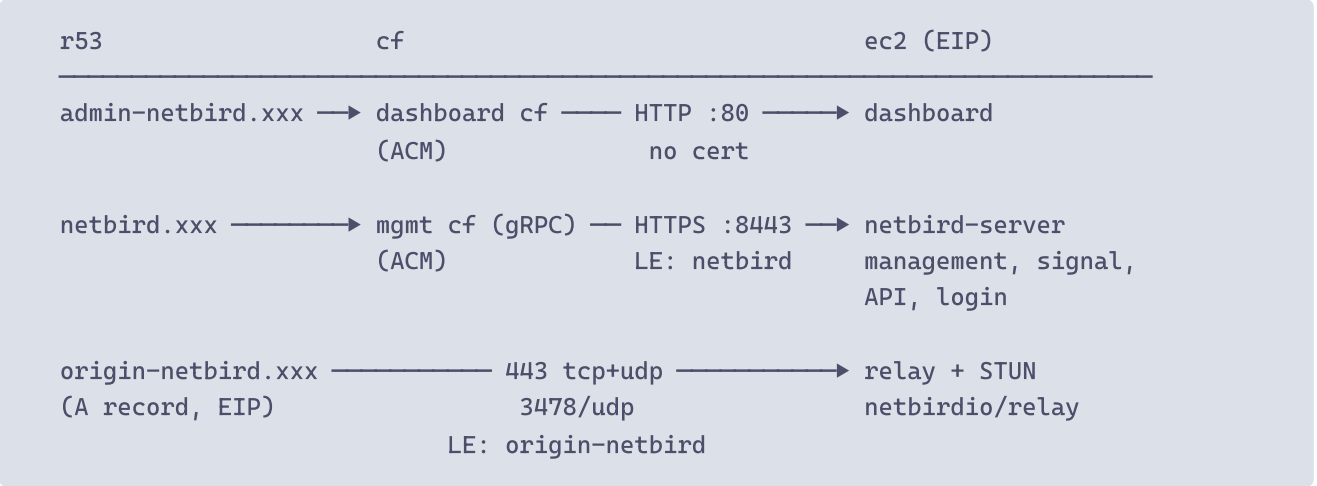
STUN (UDP, no CloudFront)
  peer ─UDP :3478, no cert─▶ EIP ─▶ STUN :3478/udp

LE: Let's Encrypt cert for netbird
cf reaches the EIP via ec2-...amazonaws.com (AWS DNS, not Route 53)
```

- 兩個 CloudFront 的 origin 都寫 `ec2-…amazonaws.com`，這是 AWS 自動產生、指向 EIP 的名字
- STUN 用自己的 `stun-netbird`（Route 53 A 記錄指 EIP），不簽憑證，STUN 是 UDP 用不到
- Let's Encrypt 只簽 `netbird`。CloudFront 會把 Host 轉給機器，AWS 文件寫回源憑證只要符合 origin 或 Host 其中一個就好。`amazonaws.com` 的名字 Let's Encrypt 不發（policy 禁止）
- relay 是 WebSocket，不是 gRPC。它留在 netbird-server，跟 gRPC 共用 443 和同一張憑證，回源那段也有加密
- 只能用 WebSocket（TCP），沒有 QUIC（UDP），掉封包時延遲會變高
- 對所有人只開 3478/udp，80、443 只收 CloudFront

- STUN 不能走 CloudFront：CloudFront 只收 HTTP、HTTPS、WebSocket，不轉 UDP。10/1 實測，CloudFront 台北節點的 3478/udp 沒回應，Cloudflare 的公開 STUN 26 ms 就回應

B：relay 直連



- 多一筆 `origin-netbird`（A 記錄指 EIP）。relay 直連要 TLS，憑證只能簽在自己的網域上
- relay 用 443（TCP 給 WebSocket，UDP 給 QUIC），netbird-server 改 8443，只收 CloudFront
- relay 有 QUIC、走直線，流量算 EC2
- 多一張 Let's Encrypt 憑證，快取也要備份到 S3，不然重開機一直重簽會撞每週上限
- 對所有人開 443 和 3478/udp

容器

- netbird-server：management、signal、API、登入，A 還包含 relay。Route 53 DNS-01 簽 Let's Encrypt
- STUN（`netbirdio/relay`，`--enable-stun`）：B 也順便負責 relay，跟 netbird-server 共用 `authSecret`
- dashboard：80
- STUN 一定要另外一個容器：合併版的 STUN 位址從 `exposedAddress` 產生，要指到別的位址只能設 `server.stuns`，一設就關掉內建的 STUN（rc1 `combined/cmd/config.go`）
- 只是同一個 ECS task 多一個容器，不是新的 service 或機器

relay 費用（美元 / 月）

EC2 額度：100 GB
CloudFront 額度：1TB

每月 relay 量	直連，免費額度還在	直連，額度已被用完	走 CloudFront，額度還在	走 CloudFront，額度已被用完
50 GB	0	5.7	3.0	9.0
200 GB	11.4	22.8	12.0	36.0
500 GB	45.6	57.0	30.0	90.0

- 直連：EC2 送出去每 GB 0.114，免費額度是每月前 100 GB
- 走 CloudFront：送給 peer 每 GB 0.12（台灣的 edge 算亞太區），免費額度是每月前 1 TB

什麼時候會走 relay

- 直連靠打洞：問 STUN 自己的外部位址，透過 signal 交換，兩邊同時送 UDP
- 兩邊都是對稱型 NAT（手機網路常見）或網路擋 UDP 就打不通
- `netbird status -d` 看是 `P2P` 還是 `Relayed`

對外開的 port

- CloudFront，只開給 CloudFront 的 port 從外面看是關的，掃描和流量攻擊由 AWS 擋
- 3478/udp（A、B 都有）
- 443（只有 B）：只有 relay，沒有有效 token 的連線會被拒絕。要注意 relay 本身的漏洞、大量連線吃光 CPU、掃描造成的 log 量
- 防護：跟著更新 image，機器對外流量和 log 寫入量加告警

待實測

- CloudFront 會不會切斷長時間的 gRPC 串流跟 WebSocket
- 回源憑證用 Host（`netbird`）比對，文件寫可以，但測測看
- relay 走 CloudFront 的速度

其他今天做的

最後選 A。下午部署時又改了回源的網域，修了 GeoLite 跟 peer IP 的問題，傍晚 commit 完就 destroy。

tcpdump 的 log 一天花掉約 300 美元

- 9/24 為了看 CloudFront 打到機器的封包，加了一個 tcpdump，把 80、443 的封包寫進 CloudWatch
- 過濾條件是 `tcp port 443`，機器自己往外連 443 也會被抓，包括把 log 上傳到 CloudWatch 的連線。上傳的流量又被抓成新的 log，自己餵自己，一天約 400 GB，CloudWatch 東京每寫入 1 GB 收 0.76 美元
- log 裡還有明文的密碼、token 跟 cookie
- 功能整個從程式裡刪掉，不留開關。以後真的要抓封包，只抓進到自己 port 的流量，而且要放進 s3 不是放進 CloudWatch

每月成本

照程式估約 14 到 16 美元，主機 spot 約 4.7，兩顆 30 GB 的系統碟各 2.9，EIP 3.7，其他接近 0。備援機如果一直開著，每月多約 19 美元。這個帳號看不到帳單，CloudFront、EC2 對外流量的免費額度是整個組織共用的。

GeoLite 下載失敗讓服務停掉

發生什麼	改法
netbird-server 每次啟動都會連 `pkgs.netbird.io` 下載 GeoLite，下載到一半被切斷就直接結束	設 `disableGeoliteUpdate: true`，開機時從 S3 還原兩個檔案（約 71 MB），每月更新一次
替補的 task 剛好排到第一次開機的備援機上，備援機接著關機，task 卡住，EIP 掛在關機的機器上	備援機第一次開機不加入 cluster

發生什麼	改法
service 最多 1 個 task，卡住的 task 擋住所有新部署，`pulumi up` 也跟著卡住	已經終止的機器在 ECS 的登記，用 `deregister-container-instance --force` 清掉

- 沒有用 `NB_DISABLE_GEOLOCATION` 整個關掉，因為還想看 peer 在哪個國家
- 官方的 `disableGeoliteUpdate` 只能避免已經有檔案還去連網，新機器沒有檔案還是會下載，所以要從 S3 放回去
- 程式需要兩個檔案，`GeoLite2-City_*.mmdb` 跟 NetBird 自己轉出來的 `geonames_*.db`，文件只寫了第一個...害我試了很久

回源改成 origin-netbird

- 原本 CloudFront 回源寫 `ec2-…amazonaws.com`，憑證靠轉給機器的 Host（`netbird`）比對
- 改成新的 `origin-netbird`，A 記錄指 EIP，Let's Encrypt 只簽這個名字，兩個 CloudFront 都回源到它
- `netbird` 那個 CloudFront 改用 `Managed-AllViewerExceptHostHeader`，不轉 Host，機器看到的 Host 跟 TLS 名字都是 `origin-netbird`

名字	憑證	誰會連
`netbird`、`admin-netbird`	ACM	瀏覽器、peer
`origin-netbird`	Let's Encrypt	只有 CloudFront

部署後檢查都通過，經過 CloudFront 的 gRPC、登入、relay 的 WebSocket、dashboard 都正常，STUN 直連有回應，機器的 443 從外面直接連不到。

peer 的 IP 顯示成 CloudFront 的 IP

- 上面待實測的第一項有答案了，CloudFront 回源預設 30 秒沒資料就切斷，Sync 跟 signal 的 gRPC 串流一直被切。回源逾時改成 60 秒 (想要在高就要去跟 aws 申請了)，client 每 60 秒重連一次，peer 之間的 WireGuard 不受影響
- NetBird 只有在連線來源列在 `server.reverseProxy.trustedPeers` 裡時才讀 `X-Forwarded-For`，設成 CloudFront 的網段
- 用臨時的 CloudFront 接 grpcb.in 實測，CloudFront 轉 gRPC 不會加 `X-Forwarded-For`，client 自己塞的假值還會原樣送到後端
- 加一個 CloudFront Function，用 CloudFront 看到的真實 IP 蓋掉 `X-Forwarded-For`。只掛在會讀 IP 的 `Login`、`Sync`、`SyncMeta` 三個 gRPC 方法，不是每個請求都跑，每月前 200 萬次免費
- 還沒部署驗證就 destroy 了

其他

- WSL 沒開 systemd，netbird 的背景服務起不來，要用 `sudo netbird up --foreground-mode --management-url ...`

小結

STUN 以外都走 CloudFront。tcpdump 抓到自己上傳的 log，一天燒掉約 300 美元，我真的很抱歉。GeoLite 下載失敗會停服務，改成開機時從 S3 放回。

參考資料

- How NetBird Works - NetBird Docs
- RFC 8489: Session Traversal Utilities for NAT (STUN)
- pcap-filter(7) man page - TCPDUMP & LIBPCAP(`port` 來源或目的其中一邊符合就算)
- Amazon CloudWatch Pricing(log 照寫入量收費)
- Management Geolocation - NetBird Docs(`disableGeoliteUpdate`、`NB_DISABLE_GEOLOCATION`)
- GeoLite Databases and Web Services - MaxMind
- Origin settings - Amazon CloudFront(Response timeout 預設 30 秒)
- Quotas - Amazon CloudFront(Response timeout per origin 可以設 1 到 120 秒)
- Use managed origin request policies - Amazon CloudFront(`AllViewerExceptHostHeader`)
- Require HTTPS for communication between CloudFront and your custom origin - Amazon CloudFront(回源憑證要符合 origin 網域或轉過去的 Host)
- Request and response behavior for custom origins - Amazon CloudFront(`X-Forwarded-For`，實測 gRPC 不會加)
- CloudFront Functions event structure - Amazon CloudFront(`event.viewer.ip`)
- Amazon CloudFront Pricing - Pay as you go(Function 每月前 200 萬次免費)
- EC2 On-Demand Instance Pricing(Data Transfer)
- Use systemd to manage Linux services with WSL - Microsoft Learn